ESCOLA DE ROBÓTICA DE NOVA LIMA - SENAI

MARILENE DA SILVA ARAUJO

RELATÓRIO TÉCNICO: IMPLEMENTAÇÃO DO PADRÃO DE PROJETO OBSERVER EM

Sistema de Monitoramento de Mercado Financeiro em C# e WPF

NOVA LIMA

2026

MARILENE DA SILVA ARAUJO

RELATÓRIO TÉCNICO: PADRÃO DE PROJETO OBSERVER

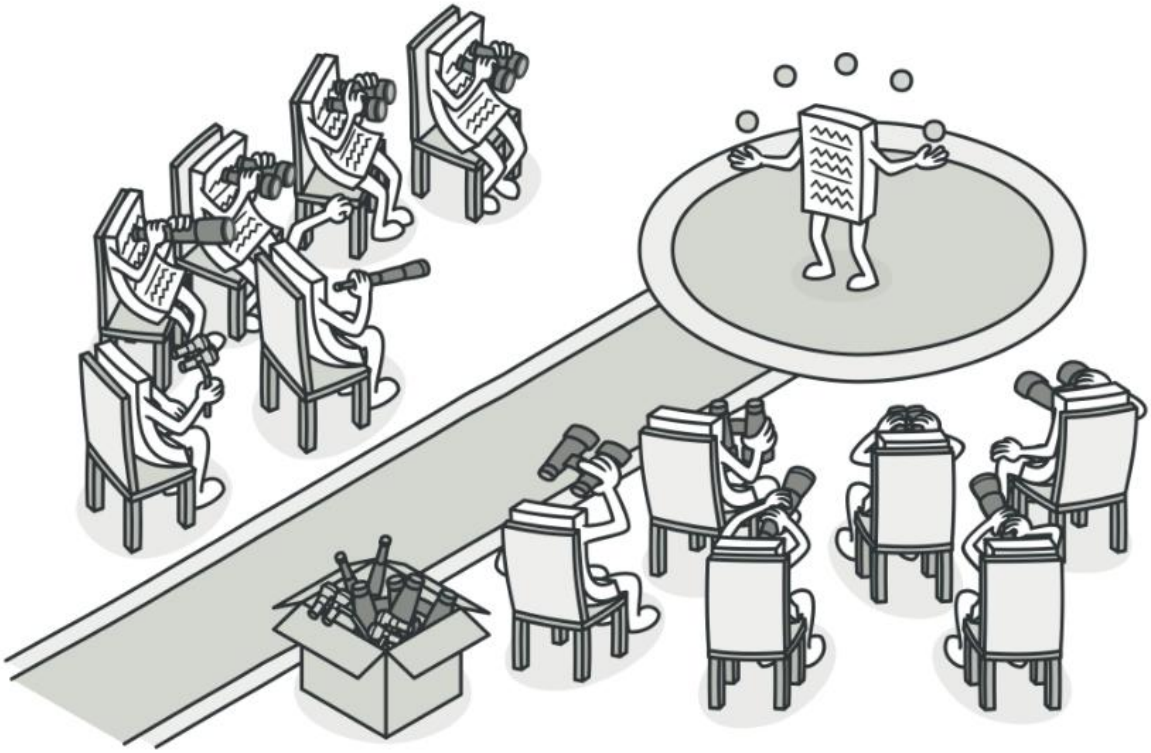
Relatório técnico apresentado como requisito parcial para avaliação na disciplina de Desenvolvimento de Sistemas / Padrões de Projeto do Curso de Desenvolvimento de Sistemas da Escola de Robótica de Nova Lima – SENAI.

Instrutor: Frederico Martins Aguiar

NOVA LIMA

2026

IMPLEMENTAÇÃO DO PADRÃO DE PROJETO OBSERVER EM SISTEMA DE MERCADO FINANCEIRO



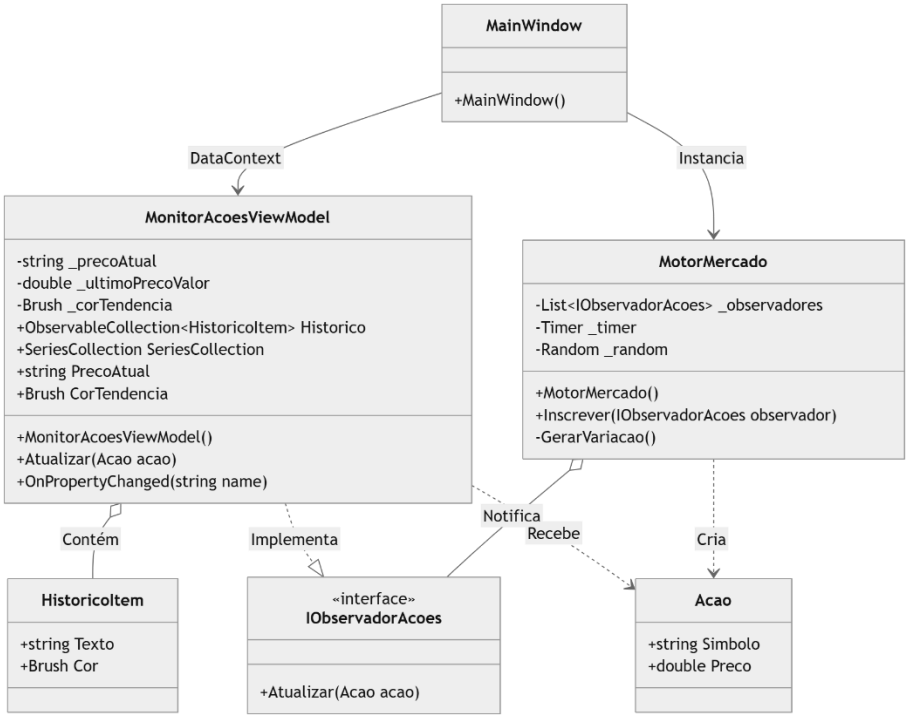
1. DEFINIÇÃO DO PADRÃO

O padrão *Observer* (Observador) é um padrão de projeto comportamental que estabelece uma relação de dependência do tipo "um-para-muitos". Segundo a definição clássica da *Gang of Four* (GoF), ele permite que um objeto, denominado **Sujeito** (*Subject*), notifique automaticamente todos os seus dependentes, chamados de **Observadores** (*Observers*), sobre qualquer mudança em seu estado interno, invocando um método padronizado de atualização.

2. PROBLEMA QUE RESOLVE

O padrão soluciona o acoplamento rígido entre objetos que precisam manter consistência de dados. Sem o *Observer*, o objeto central precisaria conhecer detalhes de todos os dependentes para notificá-los manualmente. Alternativamente, os dependentes precisariam realizar *polling* (verificação constante e repetitiva), o que resultaria em alto consumo de processamento, latência na informação e um código de difícil manutenção.

3. ESTRUTURA E PARTICIPANTES



A estrutura baseia-se em uma classe principal que gerencia uma lista de assinantes. A comunicação é realizada por meio de uma interface comum, garantindo o baixo acoplamento, onde o Sujeito não depende de classes concretas.

- **Sujeito (MotorMercado):** Mantém a lista de observadores e fornece métodos para inscrever ou remover assinantes. No Diagrama de Sequência, é ele quem dispara o loop de variação de preços.
- **Interface do Observador (IObservadorAcoes):** Define o contrato de comunicação, especificamente o método Atualizar(), permitindo que múltiplos tipos de observadores possam ser notificados.
- **Observador Concreto (MonitorAcoesViewModel):** Implementa a interface e reage às mudanças. Conforme o diagrama de sequência, após receber a atualização, ela executa processos internos como Atualiza PrecoAtual e OnPropertyChanged().
- **Objeto de Dados (Acao):** Entidade (DTO) que encapsula as informações alteradas (símbolo e preço) para transporte seguro entre o Sujeito e o Observador.

4. JUSTIFICATIVA DA ESCOLHA

A seleção do Mercado Financeiro como cenário para este projeto fundamenta-se na natureza intrinsecamente volátil e crítica dos dados econômicos. Em sistemas de monitoramento de ativos e cotações, a integridade e a tempestividade da informação são vitais; qualquer atraso (*latency*) no processamento pode resultar em decisões equivocadas e prejuízos financeiros reais.

A implementação do padrão Observer justifica-se por:

- **Reatividade em Tempo Real:** Garante que a Interface do Usuário (UI) atue como um reflexo fiel, imediato e sincronizado do motor de processamento de dados.
- **Eficiência de Recursos:** Ao contrário de modelos baseados em consultas repetitivas (*polling*), o motor "empurra" (*push*) o dado para a interface apenas quando ocorre uma variação real, otimizando o consumo de CPU.
- **Separação de Responsabilidades (MVVM):** Facilita a manutenção ao isolar a lógica de geração de preços da lógica de apresentação visual, permitindo que o sistema seja expandido sem comprometer a estabilidade do núcleo de dados.

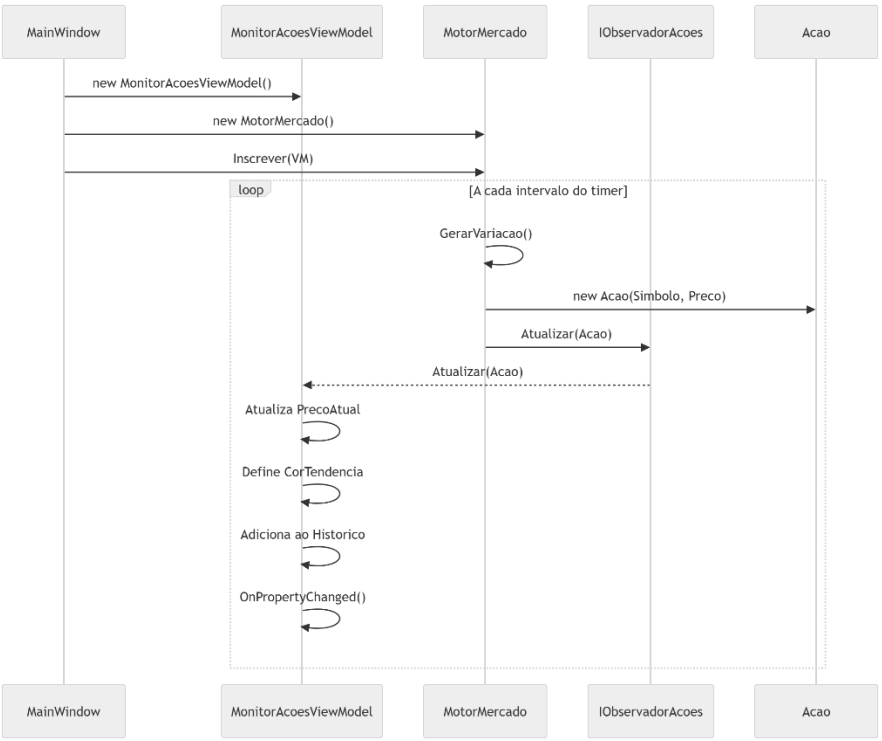
5. EXPLICAÇÃO DA IMPLEMENTAÇÃO NO PROJETO

A implementação prática no projeto "Monitor de Mercado Financeiro" foi estruturada para garantir a separação total entre a geração de dados e a sua visualização. A lógica de execução segue um ciclo reativo, detalhado conforme as etapas abaixo:

5.1. Arquitetura e Componentes

O sistema utiliza a classe MotorMercado como o Sujeito central. Esta classe possui um System.Timers.Timer que simula a volatilidade do mercado financeiro de forma assíncrona. A cada ciclo do cronómetro, o motor executa o método GerarVariacao(), que cria uma nova instância da classe Acao contendo o símbolo do ativo e o seu preço atualizado.

5.2. Dinâmica do Fluxo de Dados (Step-by-Step)



De acordo com a modelagem de sequência do projeto, o fluxo operacional compreende:

1. **Inicialização e Registo:** Ao iniciar a aplicação, a MainWindow instancia a MonitorAcoesViewModel e o MotorMercado. Em seguida, a ViewModel é registada na lista de observadores do motor através do método Inscrever(VM).

2. **Notificação Reativa (*Push*):** Quando uma nova cotação é gerada, o MotorMercado percorre a sua List<IObservadorAcoes> e invoca o método Atualizar(Acao), enviando o objeto de dados para todos os assinantes simultaneamente.

3. **Processamento e Sincronização de Threads:** Ao receber a notificação, a MonitorAcoesViewModel processa a informação (calcula a tendência, define a cor e atualiza o histórico).

4. **Atualização da Interface (WPF):** Devido à natureza assíncrona do motor, a ViewModel utiliza o Dispatcher para garantir que a atualização dos dados ocorra na *Thread* principal da interface gráfica. O ciclo encerra-se com o disparo do evento OnPropertyChanged(), que reflete a mudança no ecrã automaticamente via *Data Binding*.

6. ANÁLISE CRÍTICA

A implementação do padrão *Observer* no monitor de mercado financeiro permitiu uma avaliação profunda sobre o impacto da arquitetura no ciclo de vida do software. Os resultados observados destacam pontos de alta eficiência, mas também exigem atenção a detalhes de implementação.

6.1. Escalabilidade e Extensibilidade

A aplicação demonstrou que o padrão facilita a escalabilidade horizontal do sistema. Devido ao baixo acoplamento, é possível adicionar novos observadores — como um módulo de geração de logs em arquivo, um sistema de alertas sonoros ou um serviço de notificações por e-mail — de forma simultânea e independente. Esta expansão ocorre sem a necessidade de alterar ou recompilar o código-fonte do MotorMercado, respeitando o Princípio Aberto/Fechado (*Open/Closed Principle*) do SOLID.

6.2. Desafios de Gerenciamento de Memória

Um ponto crítico identificado na análise é o gerenciamento do ciclo de vida dos objetos no ambiente .NET. Como o Sujeito mantém referências para os seus inscritos na List<IObservadorAcoes>, os observadores que não realizam o cancelamento da assinatura (*Unsubscribe*) ao serem descartados permanecem "vivos" para o *Garbage Collector*. Esta condição pode causar vazamentos de memória (*memory leaks*), sendo imperativo garantir que o método de desinscrição seja invocado no encerramento dos componentes de interface.

6.3. Sincronização e Performance

A abordagem reativa (*Push*) provou ser superior ao modelo de consulta (*Polling*) para o contexto financeiro, garantindo que a atualização da interface seja um reflexo imediato da volatilidade do mercado. Contudo, a ordem de notificação dos observadores é indeterminada, o que deve ser considerado caso existam dependências de precedência entre diferentes módulos inscritos.

7. COMPARAÇÃO ENTRE ABORDAGENS: COM E SEM O PADRÃO OBSERVER

A eficácia da arquitetura proposta pode ser melhor compreendida ao contrastá-la com um modelo de desenvolvimento convencional de acoplamento rígido.

7.1. Abordagem Convencional (Sem o Padrão)

Em uma estrutura sem a aplicação do *Observer*, o MotorMercado (lógica de negócio) precisaria possuir referências diretas e instâncias concretas da MainWindow ou de outros componentes de interface.

- **Rigidez e Fragilidade:** Qualquer alteração na camada de apresentação exigiria modificações no núcleo do motor de dados.
- **Violação do SRP:** O motor acumularia a responsabilidade de gerenciar componentes visuais, violando o Princípio de Responsabilidade Única (*Single Responsibility Principle*).
- **Dificuldade de Expansão:** Caso uma nova tela de monitoramento ou um sistema de alertas fosse criado, o código do motor precisaria ser editado e recompilado para incluir o novo destino da notificação.

7.2. Abordagem Reativa (Com o Padrão Observer)

Com a implementação do padrão, o acoplamento deixa de ser voltado para uma implementação específica e passa a ser orientado a uma Interface (IObservadorAcoes).

- Independência Tecnológica: O motor torna-se um componente autônomo, desconhecendo a identidade ou a natureza técnica dos seus observadores.
- Testabilidade: O sistema permite a realização de testes unitários isolados no motor de dados, independentemente da existência ou do funcionamento de uma interface gráfica.
- Extensibilidade Fluida: Novos observadores podem ser conectados ao fluxo de dados em tempo de execução, bastando que implementem o contrato definido pela interface, sem qualquer impacto no código-fonte do Sujeito.

8. VANTAGENS E DESVANTAGENS DO PADRÃO

A aplicação do padrão *Observer* no sistema de monitoramento financeiro trouxe benefícios estruturais significativos, embora apresente desafios intrínsecos que devem ser gerenciados pela equipe de desenvolvimento.

8.1. Vantagens

- **Respeito ao Princípio Aberto/Fechado (OCP):** O sistema permite a inclusão de novos observadores (como módulos de log ou auditoria) sem a necessidade de modificar o código-fonte do MotorMercado, garantindo a integridade do núcleo do sistema.
- **Comunicação Dinâmica:** A arquitetura suporta a inscrição e desinscrição de componentes em tempo de execução, permitindo que o sistema se adapte dinamicamente às interações do usuário ou a eventos de rede.
- **Desacoplamento entre Camadas:** Ocorre uma redução drástica da dependência entre a Interface de Usuário (UI) e a Lógica de Negócio. O motor de dados interage apenas com uma interface abstrata, o que simplifica a manutenção e a evolução isolada de cada camada.
- **Reatividade (*Push-based*):** Melhora a performance global ao eliminar o desperdício de ciclos de CPU com verificações constantes (*polling*), enviando dados apenas quando há mudanças reais de estado.

8.2. Desvantagens e Riscos

- **Indeterminação na Ordem de Notificação:** Por definição técnica, os observadores são notificados de forma independente. Caso o sistema exija uma sequência específica de processamento entre diferentes módulos, o padrão *Observer* sozinho pode não ser suficiente, exigindo uma camada de orquestração adicional.
- **Risco de Vazamento de Memória (*Memory Leaks*):** No ecossistema .NET, se o método de desinscrição (Unsubscribe) não for rigorosamente utilizado, o Sujeito manterá referências fortes aos observadores, impedindo que o *Garbage Collector* libere a memória de janelas ou componentes já fechados.
- **Complexidade de Rastreamento:** Em sistemas de grande porte, com centenas de observadores simultâneos, pode tornar-se difícil rastrear o fluxo de execução e realizar a depuração (*debug*), uma vez que o disparo das atualizações ocorre de forma desacoplada e muitas vezes assíncrona.

9. EXEMPLOS REAIS DE USO NO MERCADO

O padrão *Observer* é um dos alicerces das arquiteturas modernas, servindo como base para sistemas distribuídos e interfaces reativas de alta escala. Abaixo, detalhei os cenários reais de maior relevância na indústria tecnológica:

9.1. Frameworks de Interface e Reatividade

A arquitetura de frameworks modernos baseia-se intrinsecamente no conceito de reatividade. No ecossistema .NET (WPF), o mecanismo de *Data Binding* utiliza o *Observer* para sincronizar a View e a ViewModel por meio da interface INotifyPropertyChanged. De forma análoga, bibliotecas como React, Vue e Angular utilizam este padrão para monitorar mudanças no estado da aplicação e disparar automaticamente a re-renderização dos componentes dependentes.

9.2. Redes Sociais e Sistemas de Notificação

Plataformas como Instagram, X (Twitter) e LinkedIn operam sob a lógica de seguidores e seguidos, que é uma implementação direta da relação Sujeito/Observador. No momento em que um perfil (Sujeito) publica novo conteúdo, o sistema propaga esse evento para o *feed* de todos os usuários inscritos (Observadores), garantindo a entrega da informação em tempo real em larga escala.

9.3. Sistemas de Mensageria e Event-Driven Architecture

Sistemas de mensageria de alta performance, como Apache Kafka e RabbitMQ, utilizam o modelo *Publish/Subscribe* (Pub/Sub), que é uma evolução distribuída do padrão *Observer*. Nestes ambientes, os *Message Brokers* atuam como o Sujeito central que distribui eventos para múltiplos serviços consumidores, permitindo o desacoplamento total entre produtores de dados e sistemas de processamento.

9.4. Monitoramento de Infraestrutura (DevOps)

Ferramentas de observabilidade como Zabbix, Grafana e Prometheus monitoram a saúde de servidores e clusters. Caso uma métrica crítica como o uso de CPU acima de 90% seja atingida, o motor de monitoramento atua como Sujeito e dispara alertas para múltiplos observadores simultâneos, como canais de comunicação (Slack/Discord), e-mails de suporte e sistemas de escalonamento automático (*Auto-scaling*).

10. REFERÊNCIAS BIBLIOGRÁFICAS

GAMMA, Erich *et al*. Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA: Addison-Wesley, 1994. Acesso em: 24 mar. 2026.

REFACTORING GURU. Observer. Disponível em: https://refactoring.guru/design-patterns/observer. Acesso em: 24 mar. 2026.